

Randoop e jqwik aplicados à classe Fraction

Fernando Kunst, Eduardo Hoffmann e Lucas Mocelin

Verificação e Validação de Software — PUCRS — T1 — 2026

Problema e objetivo

Classe Fraction

Representa frações com numerador e denominador long

Normalização automática

```
Fraction.of(2, 4)    → 1/2  
Fraction.of(1, -2)   → -1/2  
Fraction.of(0, 5)    → 0/1
```

Denominador sempre positivo · fração irredutível · zero como 0/1

Objetivo

Comparar o que diferentes técnicas de teste detectam no **mesmo sistema**

- Testes por exemplos manuais
- Randoop — geração automática com contratos
- jqwik — teste baseado em propriedades

Artigos e técnicas

Pacheco et al. (2007) — Randoop

- Monta sequências de chamadas; o resultado de cada execução orienta a próxima
- Contratos funcionam como oráculo

Fink e Bishop (1997) — Propriedades

- Propriedades como especificação e oráculo
- Usa fatiamento e contextos iterativos para cobertura

No projeto

- Randoop com contratos
- jqwik adapta propriedades como oráculo
- Sem TASPEC, fatiamento ou contextos iterativos

Aplicação: contrato e propriedade

Contrato C7 (representação simplificada)

O denominador é sempre positivo

```
{
  "name": "getDenominator",
  "description":
    "C7: o denominador é sempre positivo",
  "condition": "result > 0"
}
```

Trecho simplificado da propriedade P4

Comutatividade da adição

```
@Property
void pbP4_commutativity(
    @ForAll("small") Fraction a,
    @ForAll("small") Fraction b) {
    assertEquals(a.plus(b), b.plus(a));
}
```

Valores pequenos ($|n| \leq 1000$, $1 \leq d \leq 1000$): propriedades algébricas sem overflow

P1, P10d e P13: faixa completa de long, comparando com BigInteger

Experimentos

15

Testes por exemplos

18

Métodos de propriedades
jqwik

1.467

Testes de regressão
Randoop

1.500 testes na versão correta — sem falhas ou erros

Os 18 métodos de propriedades podem executar até 1.000 tentativas cada; as tentativas não entram no total.

Configuração Randoop

- 60 segundos por execução
- Semente principal 42; sementes 1, 2 e 3 também para M1 e M4

`--npe-on-non-null-input=ERROR`

Ambiente e mutações

- JDK 21 no macOS; código compatível com Java 17
- 5 mutações, uma de cada vez: 4 defeitos + 1 alteração que preserva o contrato

Resultados das mutações

Mutação	Exemplos	jqwik	Randoop
M1: sinal incorreto	Detectou	Detectou	Detectou — 18 testes
M2: sem redução por mdc	Detectou	Detectou	Detectou — 16 testes
M3: overflow na comparação	Não detectou	Detectou	Detectou — 4 testes
M4: overflow aritmético ignorado	Detectou	Detectou	Não detectou
M5: hashCode só do numerador	Não detectou	Não detectou	Não detectou

Randoop: testes reveladores de erro que falharam com semente 42.

M3: no jqwik, só a P10d (faixa ampla) detectou. **M4:** P13 compara com BigInteger.

M5: preserva o contrato de igualdade/hashCode; não representa um defeito.

Conclusão e limitações

O que observamos

As técnicas verificam aspectos diferentes

O jqwik detectou os 4 defeitos; exemplos e Randoop, parte deles

O resultado dependeu dos oráculos e das entradas

Limites

Uma classe pequena, poucas mutações e poucas sementes; sem superioridade geral

Testes passando não provam ausência de defeitos

P13 confere os resultados retornados, mas aceita ArithmeticException sem provar que era necessária

github.com/theeduardohoffmann/fraction-lab